Recursion

Abstract Data Structures

- **Stack** –> Last in – First out
- **Queue** –> First in – First out
- **Deque** –> a combination of a queue and a stack
- **bag** –> unordered set of elements (with duplicates)
- **set** –> unordered set of elements (no duplicates)

Stacks and Lists

Linked List:

Push is accomplished using Insert Before (Prepend)
Pop is accomplish using List Delete
Size is accomplished using List Traversal
peek is accomplished by returning the first node
isEmpty() is accomplished by checking if head is null.

Array List:

Push is accomplished adding shifting all the elements to the right. Allocate more storage if necessary.
Pop is accomplished by shifting all elements left.
Size is accomplished using List Traversal
peek is accomplished by returning the first value in the array
isEmpty() is accomplished by checking if head is null.

Array based lists

An array-based list supports the common list ADT operations, such as append, prepend, insert after, remove, and search.



Recursion

What is recursion?

When something is specified in terms of *itself*.

WHY LEARN RECURSION

- Represents a new mode of thinking.
- Provides a powerful programming paradigm.
- Enables reasoning about correctness.
- Gives insight into the nature of computation

MANY COMPUTATIONAL ARTIFACTS ARE *NATURALLY* SELF-REFERENTIAL

- File system with folders containing folders.
- Fractal graphical patterns.
- Divide-and-conquer algorithms (stay tuned).

Thinking Recursively

The problem-solving techniques we have employed so far fall under the heading of **classical *iteration***, also known as the ***iterative approach***.

Iteration (Iterative)

A programming technique in which you describe actions to be **repeated using a loop** (for-loop, for-each loop, while loop, do-while loop, etc.)

Recursion (Recursive)

A programming technique in which you describe action to **be repeated using a method that calls itself.**

A non-programming example

Think about it: If you're standing in line, you might wonder what position you're in. Are you number 10 in the line? Number 20? How would you find out?

Iterative approach

- Counting the people in the line.
- This approach is like a `while` loop that continues while there are more people left to count.

Problem

- People who are taller than you
- Not be able to see all people in the front
- The line goes around the block – can't see people around the corner

Can you think of another way to determine your position in the line? Try to think about this problem ***recursively.***

Recursion - Example

Imagine that all people standing in line **work together** to solve the problem:

- Instead of having one person do all of the counting, **each person is responsible for a little piece.**

You can ask the person in front of you what place *he* or *she* is in line.

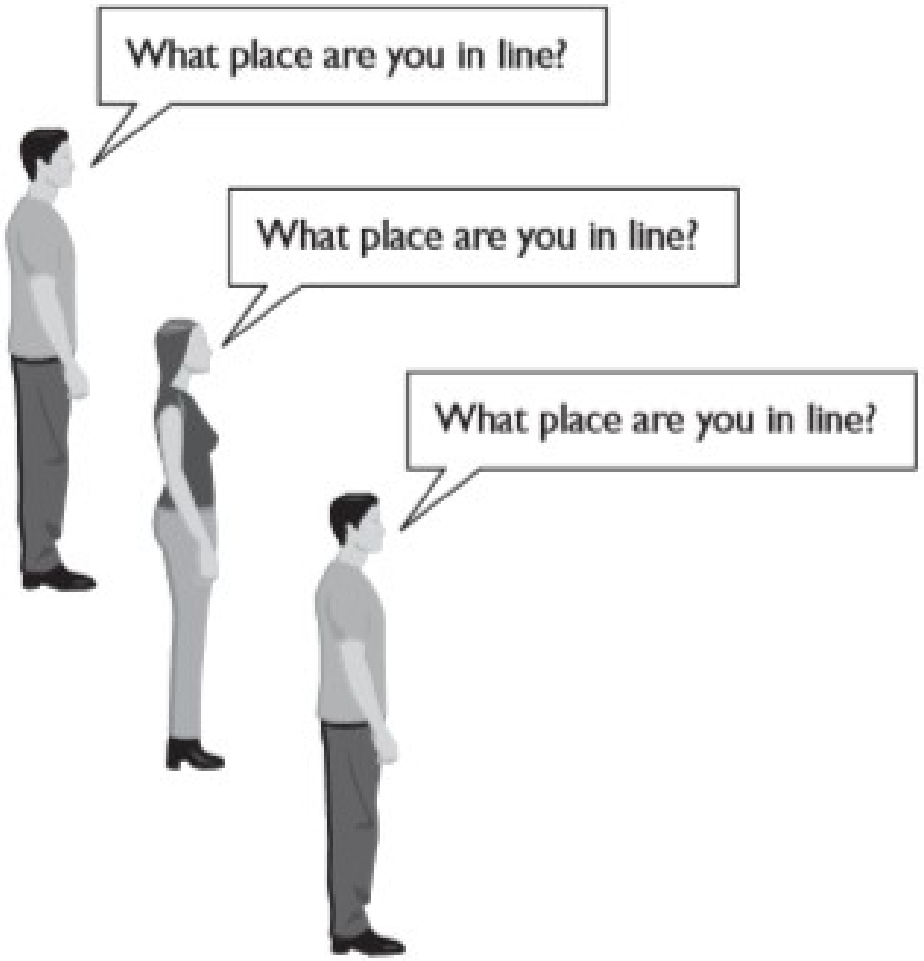


Recursion - Example

Notice that you're asking the person in front of you to think about the **exact same kind of question** you're considering: You're both trying to figure out your places in line.

The problem ***recurs*** because each of you wants to answer the same question.

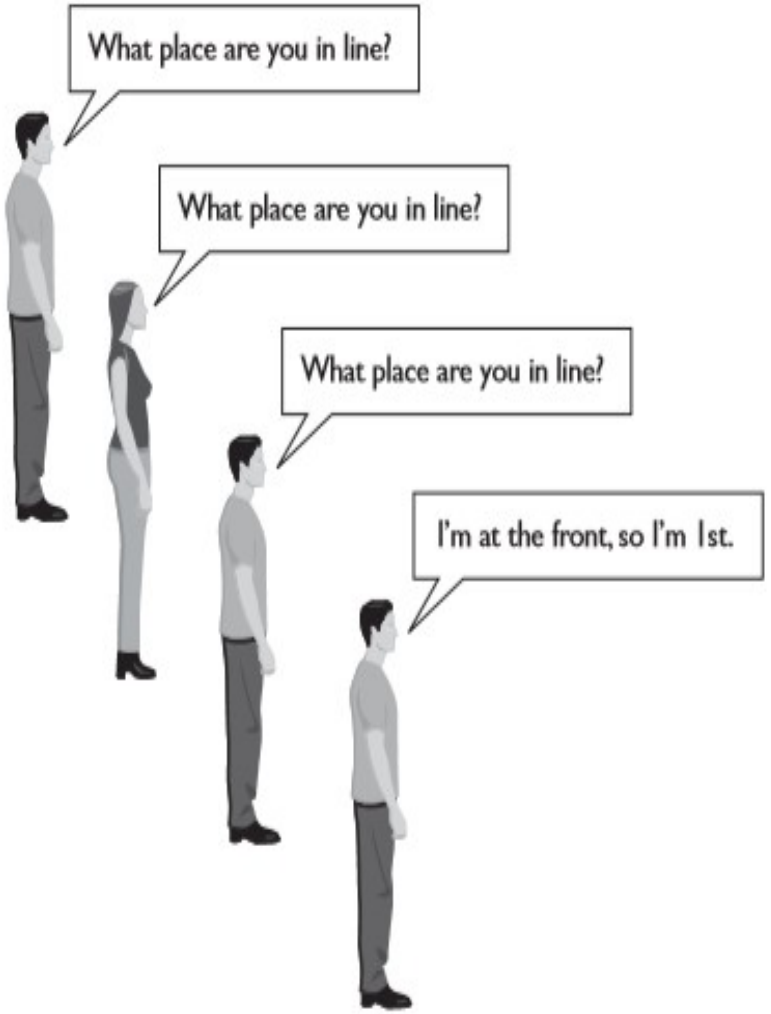
The idea is to set up a chain reaction of people, all asking the person in front the same question:



Recursion - Example

This process **has to end eventually** – when someone asks the first person in line.

At this point, you've reached the *bottom* of recursion. Now, you can start to assemble the answer.



Recursion - Example

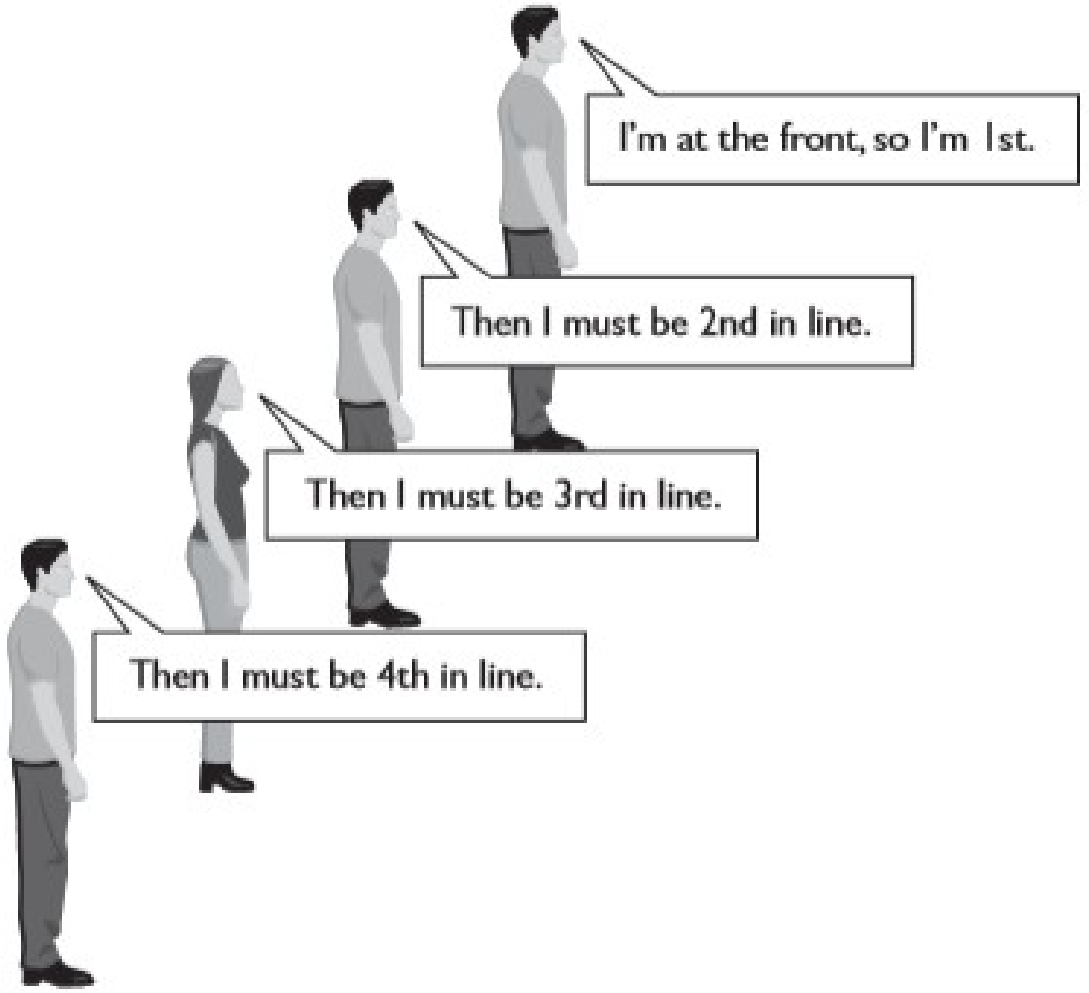
The person at the front is in positi
1,

that means the person just before
at position 2,

that means the person just before
that person is at position 3,

and so on,

…



Iterative vs Recursive

ITERATIVE SOLUTION

The iterative solution of having on person do all the counting is like having a loop that repeats some action.

RECURSIVE SOLUTION

The recursive solution of having many people each do a little bit of work translates into many different method calls, each of which performs a little bit of work.

Recursion Terms

Every recursive solution that you write will have two key ingredients: *a **base case*** and a ***recursive case***.

Base Case

A case within a recursive solution that is so simple that it can be solved directly without a recursive call.

Base case in the previous example: When the question “what place are you in line” goes to the first person in the line, the first person knows his/her position right away.

Recursive Case

A case within a recursive solution that involves reducing the overall problem to a simpler problem of the same kind that can be solved by a recursive call.

Recursive case in the previous example: When the question “what place are you in line” doesn’t reach to the first person yet, the person then needs to ask the front person “what place are you in line”.

Recursion Terms

Every recursive solution that you write will have two key ingredients: *a **base case*** and a ***recursive case***.

Base Case

A case within a recursive solution that is so simple that it can be solved directly without a recursive call.

Base case in the previous example: When the question “what place are you in line” goes to the first person in the line, the first person knows his/her position right away.

Recursive Case

A case within a recursive solution that involves reducing the overall problem to a simpler problem of the same kind that can be solved by a recursive call.

Recursive case in the previous example: When the question “what place are you in line” doesn’t reach to the first person yet, the person then needs to ask the front person “what place are you in line”.

Printing a series of Numbers

Write a *recursive method* to print the list of n numbers

```
public class VerySimpleRecursionExample {

  public static void main(String[] args) {
    callMyself(9);
  }
  /* The recursive Java method */
  public static void callMyself(long i) {
    if (i < 0) {
      return;
    }
    System.out.print(i);
    i = i - 1;
    callMyself(i);
  }
}
```

Base Case

Recursive Case

Output: 9 8 7 6 5 4 3 2 1

Factorial of a number

Write a *recursive method* to print the factorial of a given numbers

- Factorial numbers (i.e., n!) defined recursively:
  - factorial(0) = 1
  - factorial(n+1) = factorial(n) * n+1
- Examples:
  - 0! = 1
  - 1! = 1 * 1 = 1
  - 2! = 2 * 1 * 1 = 2
  - 3! = 3 * 2 * 1 * 1 = 6
  - 4! = 4 * 3 * 2 * 1 * 1 = 24

Factorial of a number

Write a *recursive method* to print the factorial of a given numbers

- Factorial numbers (i.e., n!) defined recursively:
  - factorial(0) = 1
  - factorial(n+1) = factorial(n) * n+1
- Examples:
  - 0! = 1
  - 1! = 1 * 1 = 1
  - 2! = 2 * 1 * 1 = 2
  - 3! = 3 * 2 * 1 * 1 = 6
  - 4! = 4 * 3 * 2 * 1 * 1 = 24

```
public class RecursiveJavaFactorialProgram {

  public static void main(String args[]) {
    long nFactoriral = factorialProgram(9);
    System.out.println(nFactoriral);
  }

  /* Java factorial program with recursion. */
  public static long factorialProgram(long n) {
    if (n <= 1) {
      return 1;
    } else {
      return n * factorialProgram(n - 1);
    }
  }
}
```

Alter the code to return sum of n numbers

Fibonacci Series

Write a *recursive method* to print the Fibonacci sequence of the first n numbers

The Fibonacci Sequence

1,1,2,3,5,8,13,21,34,55,89,144,233,377…

1+1=2
1+2=3
2+3=5
3+5=8
5+8=13
8+13=21
13+21=34
21+34=55
34+55=89
55+89=144
89+144=233
144+233=377

Fibonacci Series

The Fibonacci Sequence

1,1,2,3,5,8,13,21,34,55,89,144,233,377…

1+1=2	13+21=34
1+2=3	21+34=55
2+3=5	34+55=89
3+5=8	55+89=144
5+8=13	89+144=233
8+13=21	144+233=377

```
public class RecursiveFibonnaciJavaProgram {

  public static void main (String args[]) {
    for(long i=1; i<=9; i++){
      System.out.print(fibonacci(i) +" ");
    }
    System.out.println();
  }


  /* A recursive Fibonnaci sequence in Java program */
  public static long fibonacci(long number) {
    if (number == 1 || number == 2) {
      return 1;
    }
    return fibonacci(number - 1) + fibonacci(number - 2);
  }
}
```

Palindrome

Write a *recursive method* to check if the given **string** is a palindrome



Palindrome




```java
public class JavaPalindromeCheckProgram {

  public static void main(String[] args) {
    boolean flag = palindromeCheck("sitonapanotis");
    System.out.println(flag);
    flag = palindromeCheck("nine");
    System.out.println(flag);
    flag = palindromeCheck("amanaplanacanalpanama");
    System.out.println(flag);

  }
  /* Recursive Java example to check for palindromes */
  public static boolean palindromeCheck(String s){
    if(s.length() == 0 || s.length() == 1) {
      return true;
    }
    if(s.charAt(0) == s.charAt(s.length()-1)) {
      return palindromeCheck(s.substring(1, s.length()-1));
    }
    return false;
  }
}
```

Printing file contents

Write a *recursive method* called *void printFiles(dir)*that takes a directory as the parameter. Your job is to print all files in this directory and its sub-directories.

Sample input:

printFiles(a);

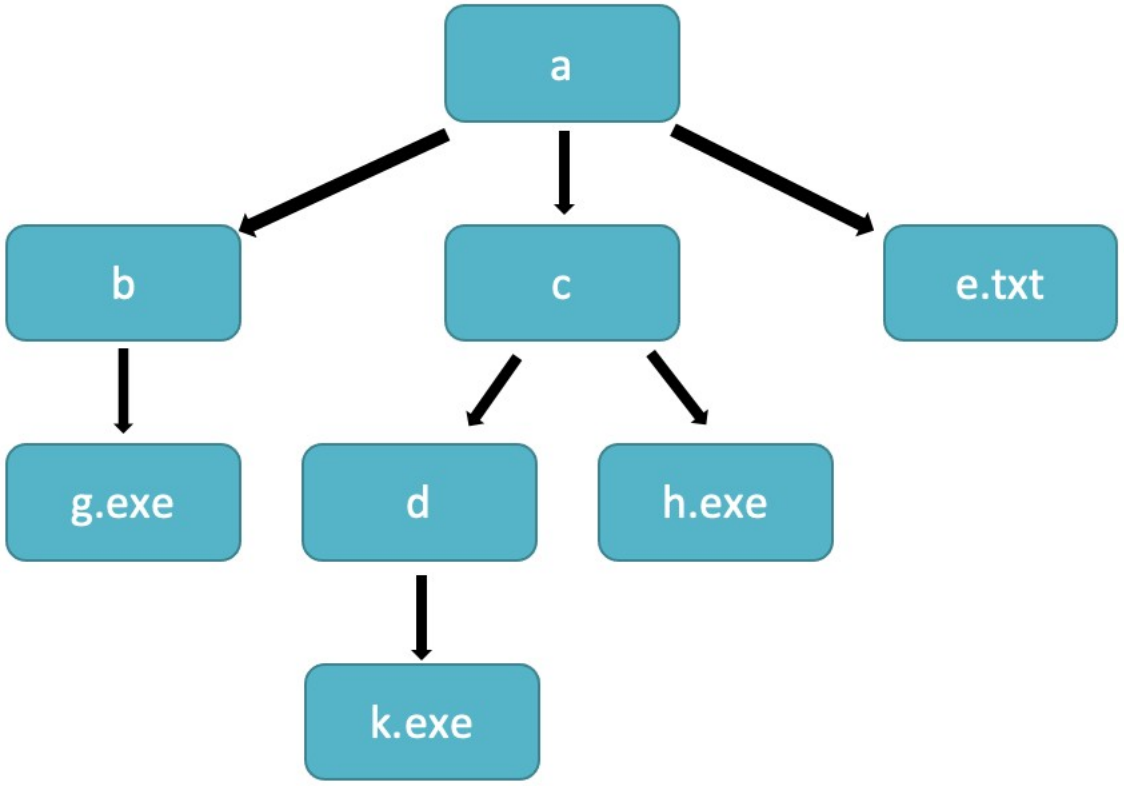
Sample output:

a/b/g.exe

a/c/d/k.exe

a/c/h.exe

a/e.txt



Recursion Practice Questions

Write a function for $mutliply(a, b)$, where a and b are both positive integers, but you can only use the $+$ or $-$ operators.

Find Greatest Common Divisor (GCD) of 2 numbers using recursion.

Write a recursive function to reverse a string. Write a recursive function to reverse the words in a string, i.e., "cat is running" becomes "running is cat".

Binary Searching

Guessing game: I'm going to randomly select an integer from 1 to 300. You'll keep guessing numbers until you find my number, and **I will tell you each time if your guess was too high, too low, or right.**

Question: What's the minimum number of guesses that guarantees to find my number?

Answer: $log_2(300) \approx 8.229$. You need no more than 9 guesses.

Why?

Binary Searching

Binary search has similar idea:

It works by *repeatedly dividing in half* the portion of the sorted list that could contain the item, until you've narrowed down the possible locations to just one.

0	1	2	3	4	5	6	7	8	9
2	7	20	21	33	41	45	48	60	67

What’s the index of 45?

In-Class Activity (Nim Game)

Consider there is pile with “n” stones. One of the player can go first and remove either 1, 2 or 3 stones. Player to remove the last stone wins. Both players are smart and rational.

Develop an algorithm for the player 2 to always win the game.

For the sake or simplicity: consider there are 12 stones and player 1 goes first. Then generalize.

Challenge yourself!

[**Hard example**] Coin game: Alice and Bob are playing a game using a bunch of coins. The players pick several coins out of the bunch in turn. Each time a player is allowed to pick 1, 2 or 4 coins, and the player that gets the last coin is the winner. Assume that both players are very smart and he/she will try his/her best to work out a strategy to win the game. For example, if there are 2 coins and Alice is the first player to pick, she will definitely pick 2 coins and win. If there are 3 coins and Alice is still the first player to pick, no matter she picks 1 or 2 coins, Bob will get the last coin and win the game. Given the number of coins and the order of players (which means the first and the second players to pick the coins), you are required to write a program to calculate the winner of the game, and calculate how many different strategies there are for he/she to win the game. You should use recursion to solve the problem, and the parameters are read from the command line. You can assume that there are no more than 30 coins.

Here are some sample runs of the program:

```
./pickcoin 1 alice bob
alice 1
./pickcoin 2 bob alice
bob 1
./pickcoin 3 alice bob
bob 2
./pickcoin 10 alice bob
alice 22
./pickcoin 25 alice bob
alice 3344
./pickcoin 30 alice bob
bob 18272
```